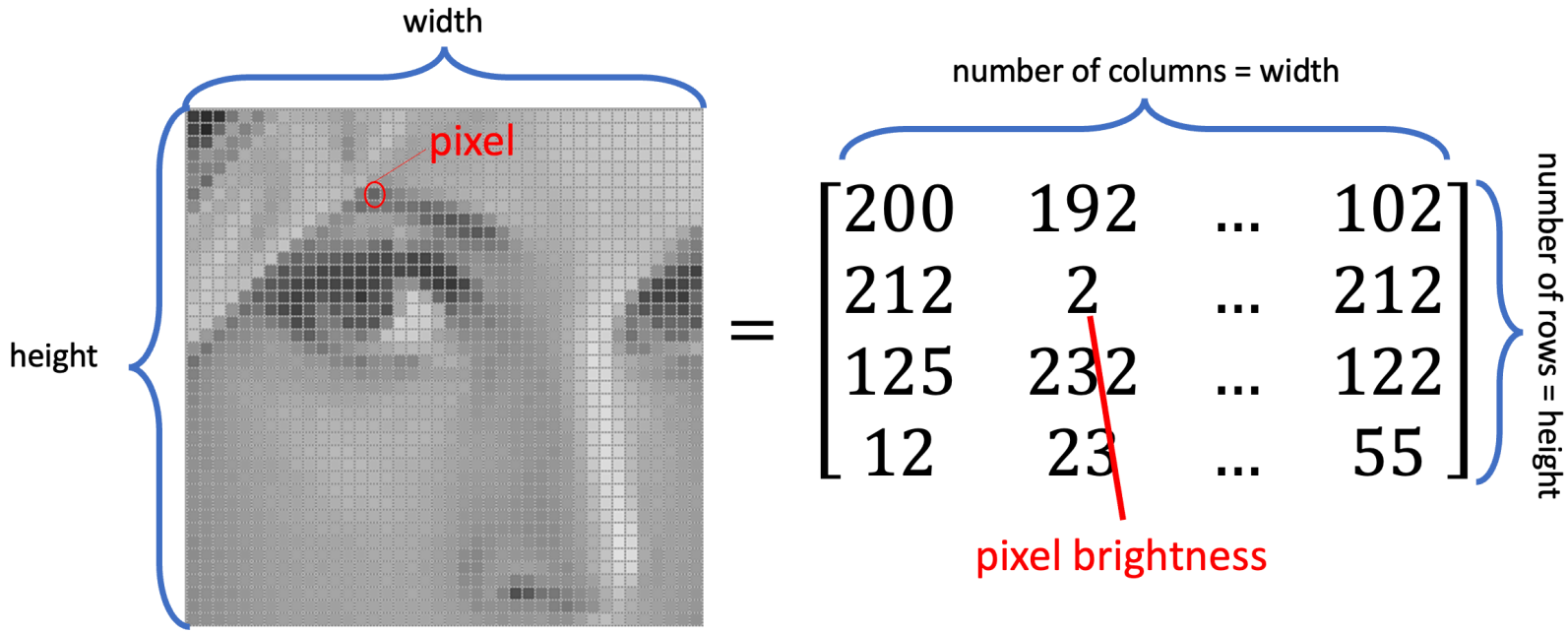
Tutorial 4: Displaying Images

Matteo Fasiolo

Images are Matrices



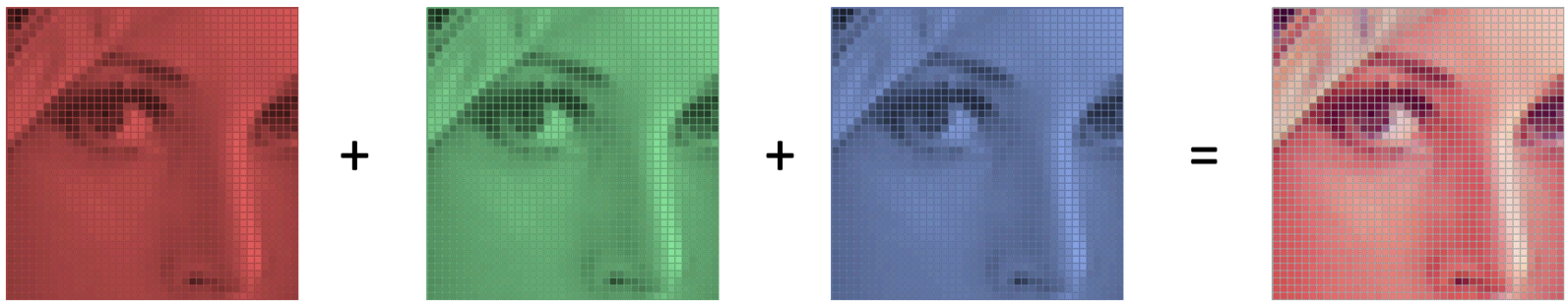
Grayscale images are expressed as matrices in computers.

A pixel in the image corresponds to an element in the matrix.

Each element of the matrix indicate the brightness of a pixel.

There are usually 256 levels of brightness, 0 is darkest while 255 is brightest.

Colored Images are Matrices too



One colored image is expressed as **three** individual matrices:

- Three matrices indicate brightness in Red, Green and Blue tones (RGB).
- Computer can display a colored image by stacking three images together.

Images Files are Flattened Matrices

Image files usually store images as flattened matrices.

- Many file system (such as tape) can only support sequential read/write.
- Recall, a row-major flattend matrix is

$$\begin{bmatrix} 1, & 2 \\ 3, & 4 \end{bmatrix} \Longrightarrow [1, 2, 3, 4].$$

Knowing these facts, we can build a "textual image viewer" using C programming language.

Building an Image Viewer

Suppose you have obtained an `int` array `a` with length `M*N`:

- It contains a flattened matrix `[12, 232, ..., 254]`
- Let the "unflattened" matrix be $A \in \mathbb{N}^{m \times n}$.
- Matrix A represents an image with width `N` and height `M`.

To visualize your image, simply print out an M by N matrix replacing the integer value of $A_{i,j}$ according to the following rules:

- if $A_{i,j} \leq 85$, print empty space .
- if $85 < A_{i,j} \leq 170$, print character `I`.
- if $170 < A_{i,j} \leq 255$, print character `M`.

Building an Image Viewer

Complete `image2char` in `image2d.c`:

1. loop over every row and column;
2. access each pixel with `image[i * Width + j]`;
3. print the corresponding character;
4. print a newline after each row.

Then uncomment the call to `image2char` in `main`.

Keep `surprise.dat` in the same folder and run the program from that folder.

Can you see the image?

If not, please zoom out by pressing ctrl + -

Optional Challenge: Detect the Edges

The current viewer treats every pixel independently.

We can produce a simple **edge image** by comparing each pixel with its four neighbours: above, below, left and right.

A pixel is an edge when its brightness differs from at least one neighbour by more than 100.

In the output image, store:

- 255 for an edge pixel;
- 0 for every other pixel.

When displayed by `image2char`, the edges will appear as `M` characters.

Optional Challenge: The Boundary

Pixels in the first or last row or column do not have four neighbours.

Set these boundary pixels to `0` without trying to access pixels outside the array.

For an interior pixel at row `i` and column `j`, its flat index is

```
int index = i * Width + j;
```

For example, the pixel above it has index `index - Width`, while the pixel to its right has index `index + 1`.

The function `abs`, declared in `<stdlib.h>`, calculates an absolute integer difference.

Optional Challenge: Your Task

Complete the following function in `image2d.c`:

```
void detect_edges(const int image[], int edges[],
                  int Height, int Width);
```

1. Visit every output pixel.
2. Set boundary pixels to `0`.
3. For an interior pixel, compare it with all four neighbours.
4. Store 255 if any absolute difference is greater than 100; otherwise store `0`.
5. Uncomment the calls to `detect_edges` and `image2char` in `main`.

What is the time complexities in terms of `Height` and `Width`?